

Projeto 4 - Equações diferenciais e o movimento oscilatório.

16 de novembro de 2023

Universidade de São Paulo - Instituto de Física de São Carlos.

Docente: José A. Hoyos

Tony Maciel Alexander. Número USP: 12624850

N.B.: Para as tarefas a seguir, foi suposto que a frequência natural de oscilação Ω_0 tem valor unitário. Logo os tempos são iguais em valor que os τ . Isso foi feito para escrever o tempo em segundos nos gráficos a seguir para ficar mais intuitivo. Porém, se a frequência natural não for unitária, a palavra "segundos" pode ser substituída por "unidades do recíproco da frequência natural".

1 Aproximação harmônica.

Problema: Resolver o pêndulo simples na aproximação harmônica pelo método de Euler e Euler-Cromer.

Pela segunda lei de Newton, a equação do pêndulo simples sem atrito é dado pela equação (1):

$$\ddot{\theta} = -\sin \theta \quad (1)$$

Onde foi usado uma mudança de variável $\omega \rightarrow \omega/\Omega$, onde $\Omega = \sqrt{g/l}$ (onde "l" é o comprimento fixo do pêndulo).

Porém, nas subseções a seguir, foi considerado que $\sin \theta = \theta$. As implicações desta aproximação será discutida, mas basicamente é responsável por variar a energia mecânica total, cujo valor será dado pela equação (2):

$$\mathcal{E} = \frac{E}{mgl} = 1 - \cos \theta + \omega^2/2 \quad (2)$$

Onde novamente foi adotado a mesma mudança de variável.

1.1 Método de Euler.

O método de Euler consiste em basicamente atualizar a posição e velocidade do pêndulo a cada instante, supondo que a velocidade angular é constante para um intervalo de tempo muito pequeno. Então, dado as condições iniciais, é considerado um instante posterior: a velocidade angular varia conforme a equação (1) (novamente, foi suposto que $\sin \theta = \theta$) e a posição varia de acordo com a velocidade angular do instante imediatamente antes. Foi empregado também o uso de derivadas numéricas. O código fonte dessa tarefa está na figura (1). Além disso, foi imposto que o ângulo não deveria ultrapassar 2π . Se isso ocorrer, o ângulo será somado por um fator de $\pm 2\pi$ para voltar-lhe no intervalo de interesse.

A posição e energia total obtidas em função do tempo estão plotadas na figura (2), juntos com a posição e energia esperadas analiticamente (foi suposto velocidade inicial nula e um ângulo inicial de 0.1 radianos). Estes serão dados por:

$$\begin{aligned} \theta(t) &= \theta_{max} \cos(t + \phi) \\ \mathcal{E} &= 1 - \cos(\theta_{max}) \end{aligned}$$

Pois $\theta(t)$ é a solução da equação (1) quando $\sin \theta = \theta$, e a energia é dada pela equação (2) quando a velocidade inicial é nula.

```

implicit none
real*8, parameter :: pi = 4*datan(1d0)
real*8, parameter :: dt = 1d-3
real*8 :: theta_i, theta_i_next, omega_i, omega_i_next, T, E_0
real*8 :: initial_theta

! supositions: initial velocity is NULL.

initial_theta = pi/2
theta_i = initial_theta
omega_i = 0
T = 0
E_0 = E(theta_i,omega_i)

open(unit=1,file="tarefa-1a-saida-3.dat")
do while (T .lt. 25*pi)
    write(1,*) T, theta_i , initial_theta*dcos(T),
&E_0, E(theta_i,omega_i)
    if (theta_i .lt. -pi) then
        theta_i = theta_i + 2*pi
    else if (theta_i .gt. pi) then
        theta_i = theta_i - 2*pi
    else
        omega_i_next = omega_i - theta_i*dt
        theta_i_next = theta_i + omega_i*dt
        omega_i = omega_i_next
        theta_i = theta_i_next
        T = T + dt
    end if
end do

close(1)

contains
! Total energy (kinetic + potential)
function E(theta, omega)
    real*8, intent(in) :: theta, omega
    real*8 :: E

    E = 1d0 - dcos(theta) + omega*omega/2d0
end function
end

```

Figura 1: Método de Euler para descobrir a posição, velocidade e energia de um pêndulo simples.

Position and energy of simple pendulum with approximation

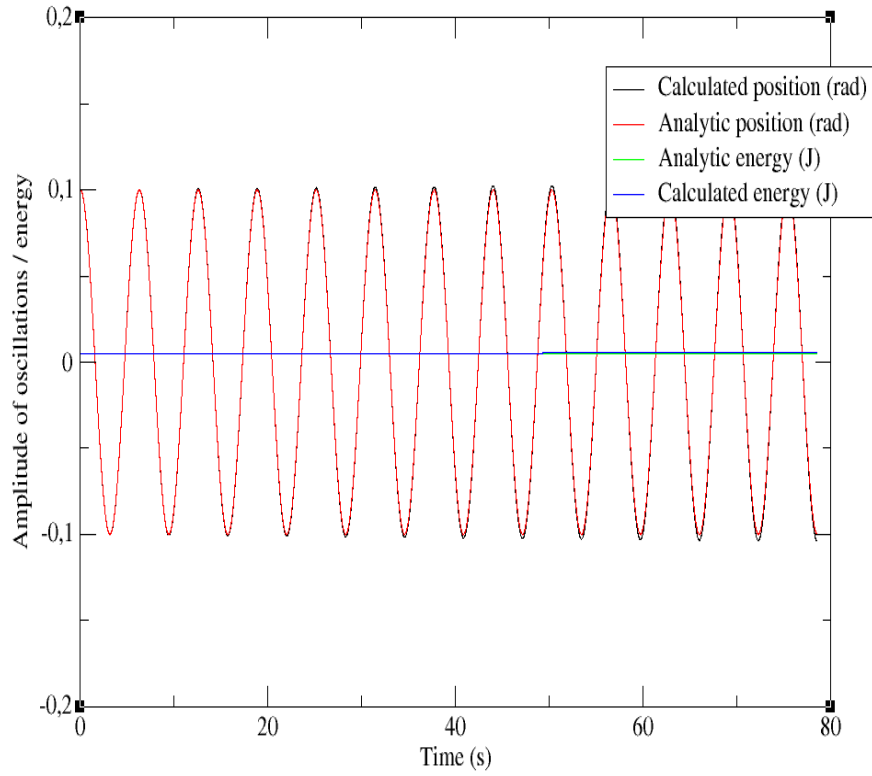


Figura 2: Posição e energia de um pêndulo simples com a aproximação $\sin \theta = \theta$, pelo método de Euler. A velocidade inicial é nula e o ângulo inicial é 0,1 radianos.

Para ver as implicações da aproximação feita, na figura (3) está a posição e energia de um pêndulo simples para um ângulo inicial de $\pi/2$ radianos e velocidade inicial nula. Pela figura, é possível notar que a energia calculada varia acima da energia analítica (teórica) calculada. Isso ocorre porque a energia analítica não representa o movimento verdadeiro do pêndulo simplificado: se não considerar que $\sin \theta = \theta$, então as energias analítica e calculada seriam iguais.

Position and energy of simple pendulum with approximation

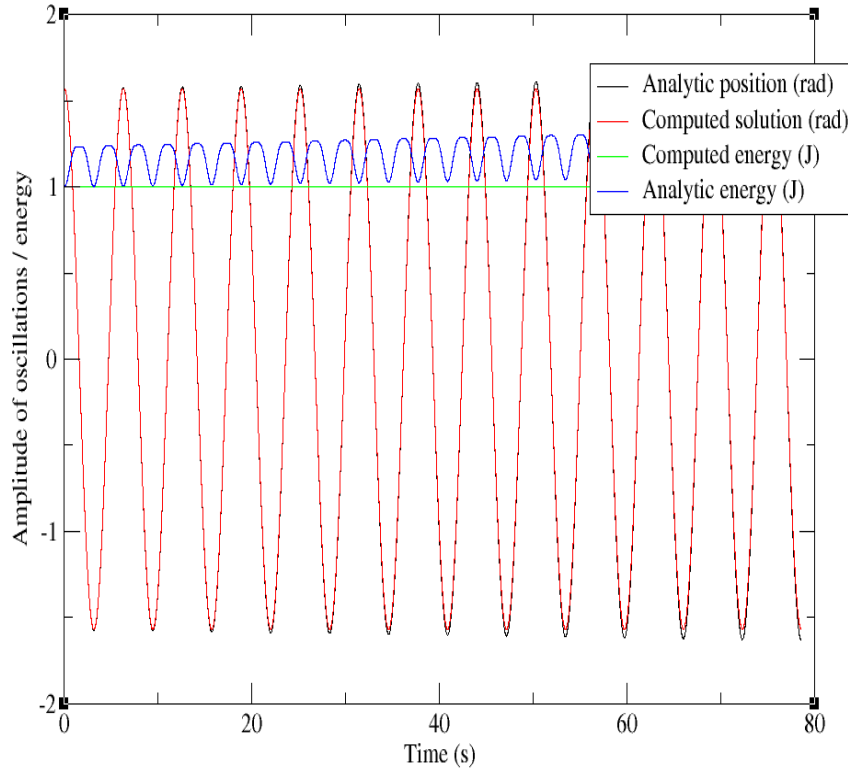


Figura 3: Posição e energia de um pêndulo simples com a aproximação $\sin \theta = \theta$, pelo método de Euler. A velocidade inicial é nula e o ângulo inicial é $\pi/2$ radianos.

Por fim, pode ser notado que em ambas figuras (2) e (3), há desacordo entre as posições calculadas e as posições analíticas (especialmente para tempos grandes). Isso ocorre por causa do erro acumulado na posição a cada iteração. Como consequência disso, com o passar do tempo a energia e amplitude acaba aumentando. Esse fenômeno está representado na figura (7).

1.2 Método de Euler-Cromer.

O método de Euler-Cromer é basicamente idêntico com o método de Euler, com uma única modificação : a velocidade angular do instante posterior é somado com a posição atual. Isso acaba mitigando efeitos cumulativos de erro.

O código fonte dessa tarefa está representada nas figuras (4) e (5) (novamente foi suposto velocidade inicial nula e ângulo inicial igual a 0.1 radianos, por efeito de comparação). Por facilidade, foi acrescentado um condicional a mais para saber qual figura o programa deve criar: a figura (6) ou a figura (7).

A saída correspondente está representado na figura (6). Comparando as

figuras (6) e (2), percebe-se que no método de Euler-Cromer, a posição calculada está mais próxima da posição analítica.

```
implicit none
real*8, parameter :: pi = 4*datan(1d0)
real*8, parameter :: dt = 1d-3
real*8 :: theta_i, theta_i_next, omega_i, omega_i_next, T, E_0
real*8 :: initial_theta, ti, nti, oi, noi
integer :: i_want_energy

print*, "energy?"
read(*,*) i_want_energy
if (i_want_energy .eq. 0) then ! make first graph

initial_theta = 1d-1
theta_i = initial_theta
omega_i = 0
T = 0
E_0 = E(theta_i,omega_i)

open(unit=1,file="tarefa-1b-saida-1.dat")
do while (T .lt. 25*pi)
write(1,*) T, theta_i , initial_theta*dcos(T),
&E_0, E(theta_i,omega_i)
if (theta_i .lt. -pi) then
theta_i = theta_i + 2*pi
else if (theta_i .gt. pi) then
theta_i = theta_i - 2*pi
else
omega_i_next = omega_i - theta_i*dt
theta_i_next = theta_i + omega_i_next*dt
omega_i = omega_i_next
theta_i = theta_i_next
T = T + dt
end if
end do
close(1)

else ! make second graph
```

Figura 4: Método de Euler-Cromer para descobrir a posição, velocidade e energia de um pêndulo simples.

```

else ! make second graph

initial_theta = 1d-1
theta_i = initial_theta
ti = initial_theta
oi = 0
omega_i = 0
T = 0
open(unit=2,file="tarefa-1b-saida-3.dat")
do while (T .lt. 100*pi)
  write(2,*) T, E(theta_i,omega_i), E(ti,oi)
  if (theta_i .lt. -pi) then
    theta_i = theta_i + 2*pi
  else if (theta_i .gt. pi) then
    theta_i = theta_i - 2*pi
  end if
  if (ti .lt. -pi) then
    ti = ti + 2*pi
  else if (ti .gt. pi) then
    ti = ti - 2*pi
  end if
  omega_i_next = omega_i - theta_i*dt
  theta_i_next = theta_i + omega_i*dt
  omega_i = omega_i_next
  theta_i = theta_i_next
  noi = oi - ti*dt
  nti = ti + noi*dt
  oi = noi
  ti = nti
  T = T + dt
end do
close(2)
end if
contains
! Total energy (kinetic + potential)
function E(theta, omega)
  real*8, intent(in) :: theta, omega
  real*8 :: E

  E = 1d0 - dcos(theta) + omega*omega/2d0
end function
end

```

Figura 5: Comparação entre as energias nos métodos de Euler e Euler-Cromer de um pêndulo simples.

Position and energy of simple pendulum with approximation

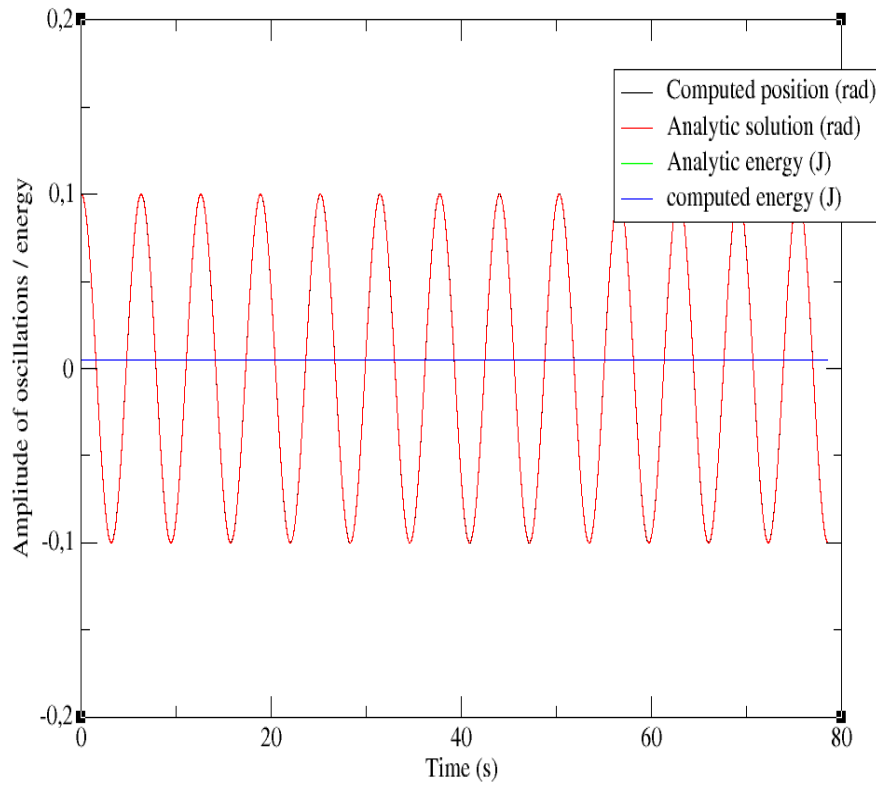


Figura 6: Posição (em radianos) e energia (em Joule) de um pêndulo simples com a aproximação $\sin \theta = \theta$, pelo método de Euler-Cromer. A velocidade inicial é nula e o ângulo inicial é 0.1 radianos.

A característica mais marcante do método de Euler-Cromer para o método de Euler está no comportamento da energia total do sistema. Na figura (7), está representado a energia calculada em ambos métodos (as oscilações ocorrem porque foi assumida a aproximação $\sin \theta = \theta$). Percebe-se um aumento sistemático da energia no método de Euler (porque o erro é mais facilmente propagado). Essa simulação foi feita rodando os métodos simultaneamente e escrevendo em um arquivo de saída o resultado.

Energy in Euler and Euler-Cromer methods

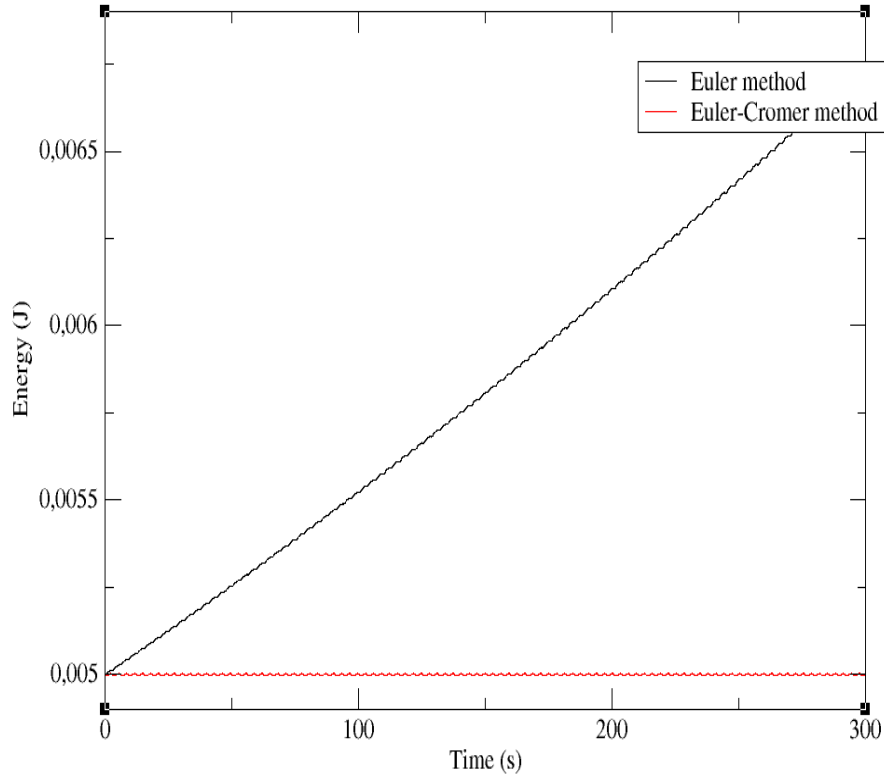


Figura 7: Energias totais em Joule de um pêndulo simples (com aproximação) pelos métodos de Euler e Euler-Cromer.

2 Período do pêndulo simples.

Problema: Calcular o período de oscilação de um pêndulo simples em função do ângulo inicial, pelo método de Euler-Cromer, integração numérica e por uma aproximação da integral para ângulos pequenos (supondo velocidade inicial nula).

Por conservação de energia, integrando a expressão resultante com respeito ao tempo, obtém-se a equação (3) para o período do pêndulo simples (sem aproximação).

$$T = \sqrt{2} \int_{-\theta_0}^{\theta_0} \frac{d\theta}{\sqrt{\cos \theta - \cos \theta_0}} \quad (3)$$

Pela substituição de variáveis $\sin u = \frac{\sin(\theta/2)}{\sin(\theta_{max}/2)}$, expandindo a expressão para um denominador pequeno, resulta na equação (4).

$$T \approx 2\pi(1 + \theta_0^2/16) \quad (4)$$

A resolução dessa tarefa foi feita de três maneiras: Euler-Cromer, integração numérica da equação (3) e pela equação (4). O código fonte dessa tarefa está nas figuras (8) e (9). Neste, foi escolhido um ângulo inicial e primeiro calculado o período pelo método de Euler-Cromer (contando quantas vezes o pêndulo passa pelas raízes da posição e depois fazendo uma média), depois foi integrado a equação (3) numericamente pelo método do trapézio com bastantes subdivisões (pois as singularidades regem um aumento de precisão) e por final, foi calculado o período pela equação (4). Em geral, os resultados estão de acordo entre si (especialmente para ângulos pequenos), conforme evidenciado pela figura (10) que registra os períodos calculados das três maneiras.

A figura (11) mostra o comportamento do período do pêndulo simples como função do ângulo inicial de lançamento, supondo uma velocidade inicial nula. O método usado para calcular os períodos dessa figura foi a integração numérica da equação (3), mas qualquer método poderia ser usado, por registrarem valores muito próximos uns dos outros. O comportamento da curva da figura (11) está de acordo com o esperado, pois quanto maior for o ângulo inicial, maior deverá ser o período.

```

implicit none ! note to self: use -Ofast to compile or intel...
real*8 , parameter :: pi = 4*datan(1d0)
real*8, parameter :: dt = 1d-5
real*8 :: theta_i, theta_i_next, omega_i, omega_i_next
real*8 :: initial_angle, T_Euler, T_approx, T_int, T
real*8 :: period, last_zero
integer :: i, oscillations
logical :: first

print*, "  Initial angle", "                                Euler-Cromer",
&"          Integral", "                                Approximation"
open(unit=1, file="tarefa-2a-saida-1.dat")
do i = 1, 100
  initial_angle = 5d-3*i*pi
  theta_i = initial_angle
  omega_i = 0 ! initial velo is 0
  oscillations = 0
  first = .true.
  T = 0
  ! have each pendulum go back and forth 10 times
  do while (oscillations .lt. 10)
    if (theta_i .lt. -pi) then
      theta_i = theta_i + 2*pi
    else if (theta_i .gt. pi) then
      theta_i = theta_i - 2*pi
    end if
    omega_i_next = omega_i - dsin(theta_i)*dt
    theta_i_next = theta_i + omega_i_next*dt
    if (theta_i * theta_i_next .le. 0) then ! theta is zero
      if (first .eqv. .true.) then ! start counting
        first = .false.
        period = T
      else
        oscillations = oscillations + 1
        last_zero = T
      end if
    end if
    omega_i = omega_i_next
    theta_i = theta_i_next
    T = T + dt
  end do
  T_int = dsqrt(2d0)*trapezoid(-initial_angle, initial_angle)
  if (mod(i, 10) .eq. 0) then ! for table
    ! mean of period. 2* because time for zeros is half period
    T_Euler = 2*(last_zero - period) / oscillations
    T_approx = 2*pi*(1d0 + initial_angle*initial_angle/16)
    print*, initial_angle, T_Euler, T_int, T_approx
  end if
  write(1, *) initial_angle, T_int ! for graph
end do
close(1)

contains

```

Figura 8: Cálculo do período de um pêndulo simples de três maneiras: pelo método de Euler-Cromer, integração numérica da equação (3) e pela fórmula (4).

```
contains

function trapezoid(a,b)
  ! it is supposed that a is negative and b is positive
  real*8, intent(in) :: a, b
  real*8 :: trapezoid , h, temp, cos_b
  real*8, parameter :: partitions = 1d8
  integer :: i

  trapezoid = 0
  h = (b - a) / partitions
  temp = a + h
  cos_b = dcos(b)
  do i = 2 , int(partitions) - 1
    trapezoid = trapezoid +1d0/dsqrt(dcos(temp) - cos_b)
    temp = temp + h
  end do
  trapezoid = trapezoid +5d-1/dsqrt(dcos(a + h/5) -cos_b)
  &+ 5d-1/dsqrt(dcos(b - h/5) - cos_b)
  trapezoid = h*trapezoid
end function
end
```

Figura 9: Função que calcula a integral da equação (3) de modo numérico pelo método do trapézio.

Initial angle	Euler-Cromer	Integral	Approximation
0.15707963267948966	6.2928879998570491	6.2925469163871020	6.2928747686421795
0.31415926535897931	6.3221639998667971	6.3218200414325256	6.3219431530299612
0.47123889803846897	6.3715159998832291	6.3711687630420686	6.3703904603429287
0.62831853071795862	6.4418179999066352	6.4414641883659511	6.4382166905810863
0.78539816339744828	6.5343439999374437	6.5339857624598237	6.5254218437444287
0.94247779607693793	6.6508639999762398	6.6504956046908417	6.6320059198329586
1.0995574287564276	6.7937200000238054	6.7933485353322922	6.7579689188466778
1.2566370614359172	6.9659960000811649	6.9656051144935063	6.9033108407855819
1.4137166941154069	7.1717420001496688	7.1713571052356890	7.0680316856496752
1.5707963267948966	7.4163000002310966	7.4158712279616550	7.2521314534389560

Figura 10: Período de um pêndulo simples em função do ângulo inicial da lançamento. A primeira coluna são os ângulos iniciais e as demais colunas são os períodos calculados pelas maneiras nos cabeçalhos.

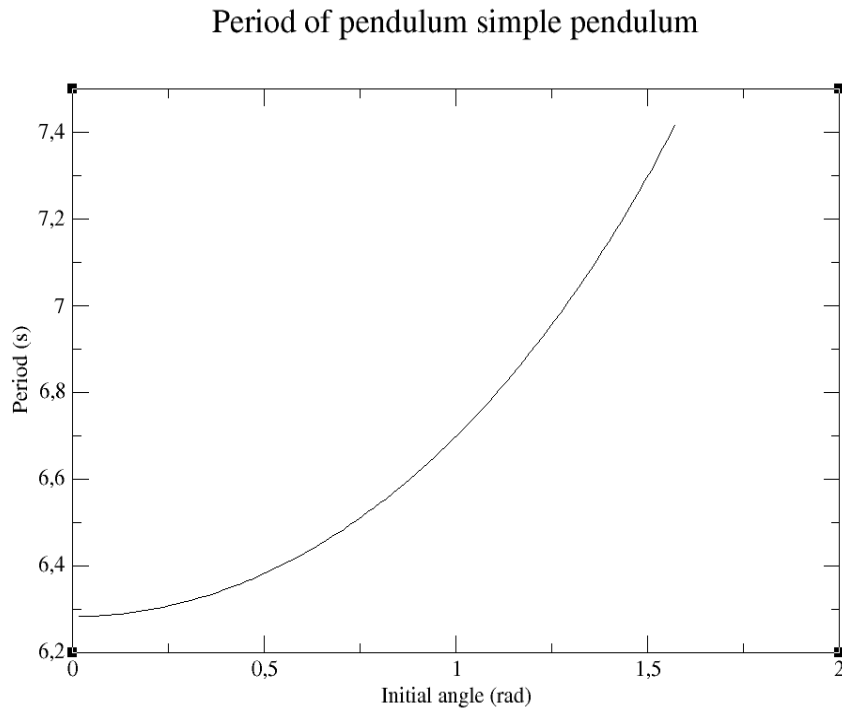


Figura 11: Período de um pêndulo simples calculado pela equação (3) numericamente em função do ângulo inicial de lançamento.

3 Pêndulo amortecido.

Problema: Resolver o pêndulo simples amortecido e indicar se é sub-,super- ou criticamente amortecido para um dado fator de amortecimento.

A equação do pêndulo amortecido é, pela segunda lei de Newton (usando a mesma mudança de variável do restante do relatório) dada pela equação (5):

$$\ddot{\theta} = -\sin \theta - \gamma \dot{\theta} \quad (5)$$

Seja o fator de amortecimento $\gamma = 1/2$. Logo, o código fonte dessa tarefa está mostrada na figura (12). Foi utilizado o método de Euler-Cromer e o fato de que $\dot{\theta} = \omega$. Também foi analisado o movimento para os ângulos iniciais de 30, 60 e 90 graus para ter uma análise mais completa. A conclusão é de que o movimento é sub-amortecido.

A figura (13) mostra graficamente a amplitude deste pêndulo amortecido para alguns ângulos iniciais.

```

1  implicit none
2  real*8 , parameter :: GAM = 5d-1
3  real*8, parameter :: dt = 1d-3
4  real*8, parameter :: pi = 4*datan(1d0)
5  real*8 :: T, theta_i(3), next_theta(3), omega_i(3), next_omega(3)
6
7  ! initial suppositions: Null velo
8  omega_i(:) = 0
9  theta_i(1) = pi/6
10 theta_i(2) = pi/3
11 theta_i(3) = pi/2
12 T = 0
13 open(unit=1,file="tarefa-3-saida-1.dat")
14 write(1,*) T, theta_i(1), theta_i(2), theta_i(3)
15 do while (T .lt. 10*pi)
16     ! 30 degrees
17     next_omega(1) = omega_i(1) - (dsin(theta_i(1)) + GAM*omega_i(1))*dt
18     next_theta(1) = theta_i(1) + next_omega(1)*dt
19     omega_i(1) = next_omega(1)
20     theta_i(1) = next_theta(1)
21     ! 60 degrees
22     next_omega(2) = omega_i(2) - (dsin(theta_i(2)) + GAM*omega_i(2))*dt
23     next_theta(2) = theta_i(2) + next_omega(2)*dt
24     omega_i(2) = next_omega(2)
25     theta_i(2) = next_theta(2)
26     ! 90 degrees
27     next_omega(3) = omega_i(3) - (dsin(theta_i(3)) + GAM*omega_i(3))*dt
28     next_theta(3) = theta_i(3) + next_omega(3)*dt
29     omega_i(3) = next_omega(3)
30     theta_i(3) = next_theta(3)
31     T = T + dt
32     write(1,*) T, theta_i(1), theta_i(2), theta_i(3)
33
34 end do
35 close(1)
36 end

```

Figura 12: Cálculo do movimento de um pêndulo simples amortecido com fator de amortecimento $\gamma = 1/2$ pelo método de Euler-Cromer.

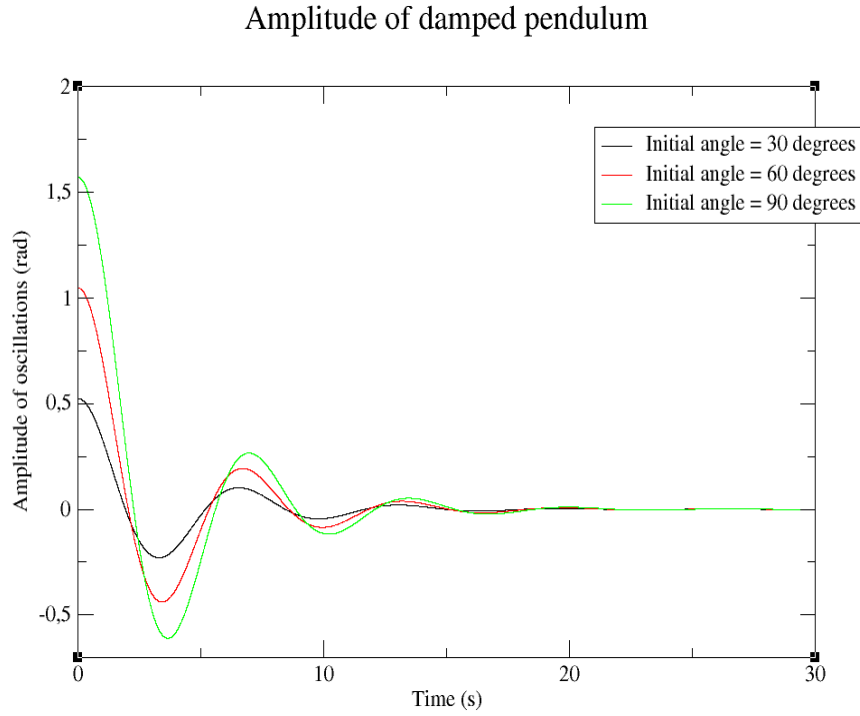


Figura 13: Amplitude do movimento de um pêndulo simples amortecido com fator de amortecimento $\gamma = 1/2$ em função do tempo para vários ângulos iniciais.

Vale a pena notar que na figura (12), o código fonte não verifica se a amplitude do movimento ultrapassa em módulo 2π , por evitar testes redundantes e melhorar com a rapidez do código, uma vez que a amplitude sempre será menor (por se tratar de um movimento amortecido, onde cada oscilação reduz a amplitude).

4 Pêndulo amortecido e forçado.

Problema: Estudar o pêndulo simples amortecido forçado através de gráficos da posição, velocidade angular, energia mecânica, seções de Poincaré e expoentes de Lyapunov.

Pela segunda lei de Newton, a equação de um pêndulo amortecido forçado com força senoidal é dado pela equação (6) (novamente supondo a mesma mudança de variáveis):

$$\ddot{\theta} = \alpha \sin(vt) - \sin \theta - \gamma \dot{\theta} \quad (6)$$

Onde α é uma constante que parametriza a intensidade do termo forçante e v é a frequência da força externa (em unidades da frequência natural). A seguir será suposto que $v = 2/3$ e $\gamma = 0.5$ para todo experimento.

4.1 Movimento geral.

A seguir serão calculadas a posição, velocidade angular e energia total de um pêndulo forçado amortecido com $\alpha = 0.5$, 0.75 , 1.0 , 1.25 , 1.4 e 1.5 pelo método de Euler-Cromer, supondo que o pêndulo parte do repouso e na posição vertical ($\theta_0 = 0$). Esses gráficos estão representadas nas figuras (16) a (21) respectivamente.

Para os casos em que $\alpha = 0.5$, 0.75 e 1.0 , o sistema é periódico após um transiente, com período aproximadamente igual a $T = 9.4247755 \approx 2\pi/\nu$. Para os demais valores, é caótico.

O código fonte dessa tarefa está representada nas figuras (14) e (15). Foi efetuado a simulação pelo método de Euler-Cromer até que o estado transiente sumisse (figura 14) e em seguida, o período do movimento foi calculado (figura 15).

Para os casos periódicos, a simulação foi feita até metade de tempo máximo de simulação para depois coletar o período na outra metade. Para os casos caóticos, a simulação foi rodada até o tempo máximo de simulação já que não terá período para calcular (na verdade o programa consta que o período é infinito, o que tecnicamente não está errado também).

Para coletar o período nos casos periódicos, foi iniciado uma contagem no primeiro momento em que a posição se anula (quando o produto entre duas posições sucessivas for menor que zero) até o último momento em que a posição se anula. Esse tempo foi dividido pela quantidade de vezes que a posição foi anulada e depois tudo foi multiplicado por 2 (porque o tempo entre uma raiz e outra é meio período).

```

implicit none
real*8, parameter :: dt = 1d-3
real*8, parameter :: gam = 5d-1
real*8, parameter :: v = 2d0/3d0
real*8, parameter :: alpha = 15d-1
real*8, parameter :: pi = dacos(-1d0)
real*8, parameter :: T_max = 100*pi
real*8 :: T, theta_i, omega_i, next_theta, next_omega
open(unit=1,file="tarefa-4b-saida-11.dat")
  T = 0
  theta_i = 0
  omega_i = 0
  do while(T .le. T_max)
    if (theta_i .lt. -pi) then
      theta_i = theta_i + 2*pi
    else if (theta_i .gt. pi) then
      theta_i = theta_i - 2*pi
    end if
    write(1,*) theta_i, omega_i
    next_omega = omega_i + (alpha*dasin(v*T) - gam*omega_i -
&dasin(theta_i))*dt
    next_theta = theta_i + next_omega*dt
    theta_i = next_theta
    omega_i = next_omega
    T = T + dt
  end do
close(1)
end

```

Figura 14: Primeira parte do código fonte da tarefa 4a (simulação de um pêndulo forçado amortecido).

```

! steady-state is achieved.
oscillations = 0
first = 1
do while (T .le. T_max)
  if (theta_i .lt. -pi) then
    theta_i = theta_i + 2*pi
  else if (theta_i .gt. pi) then
    theta_i = theta_i - 2*pi
  end if
  next_omega = omega_i + (alpha*d*sin(v*T) - gam*omega_i -
&d*sin(theta_i))*dt
  next_theta = theta_i + next_omega*dt
  if (theta_i * next_theta .le. 0) then ! theta is zero
    if (first .eq. 1) then ! start counting
      first = 0
      period = T
    else
      oscillations = oscillations + 1
      last_zero = T
    end if
  end if
  theta_i = next_theta
  omega_i = next_omega
  T = T + dt
end do
! time from 2 zeros is half period
period = 2*(last_zero - period) / oscillations
print*, period

!alpha = 0.5 , 0.75, 1.0 have period of about 2pi/v
contains
  function E(theta,omega)
    real*8, intent(in) :: theta, omega
    real*8 :: E
    E = 1 - d*cos(theta) + omega*omega/2
  end function
end

```

Figura 15: Segunda parte do código fonte da tarefa 4a (simulação de um pêndulo forçado amortecido).

Forced damped oscillator ($\alpha = 0.5$)

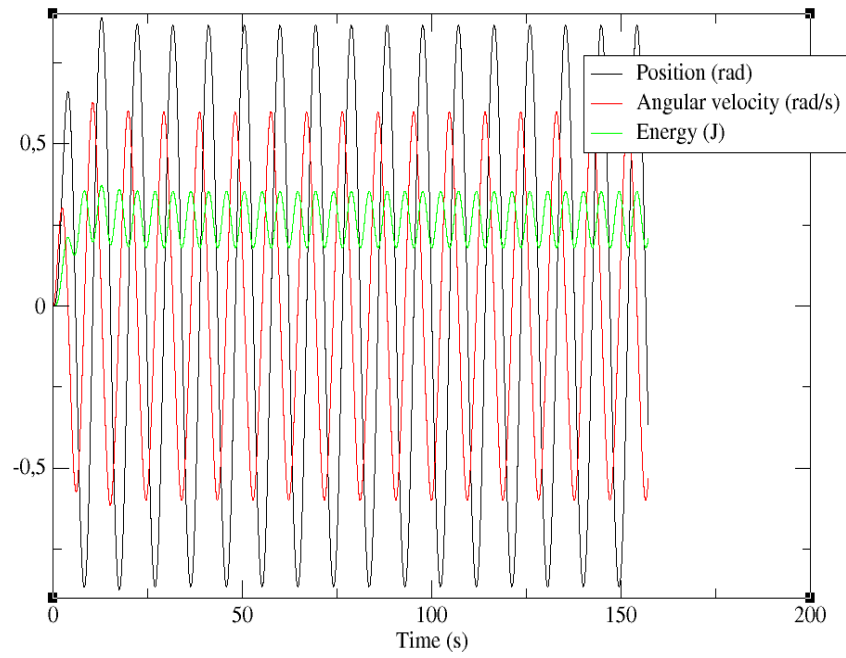


Figura 16: Posição, velocidade angular e energia de um pêndulo forçado amortecido com $\alpha = 0.5$. O sistema é periódico.

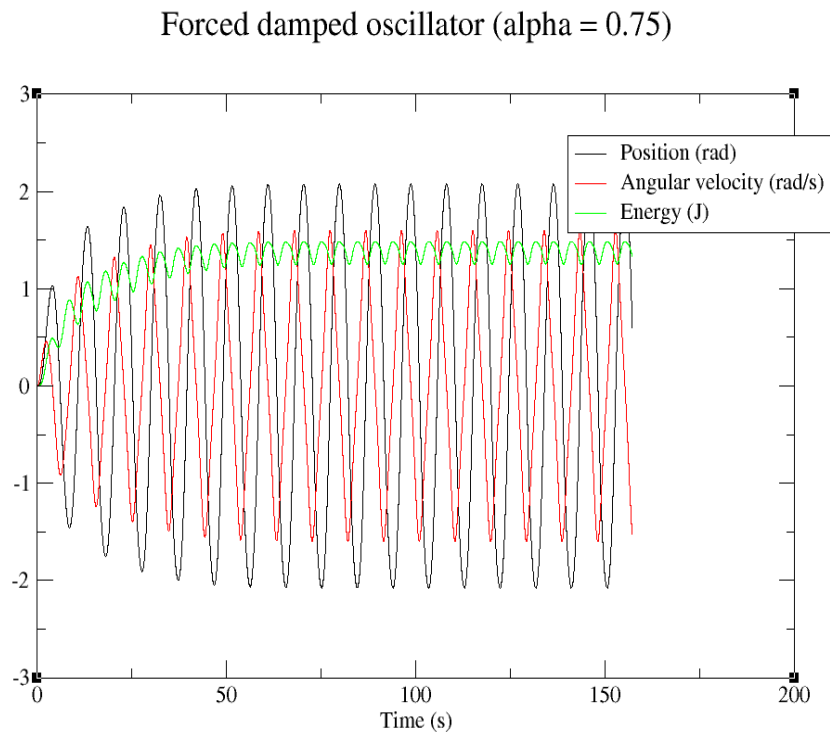


Figura 17: Posição, velocidade angular e energia de um pêndulo forçado amortecido com $\alpha = 0.75$. O sistema é periódico.

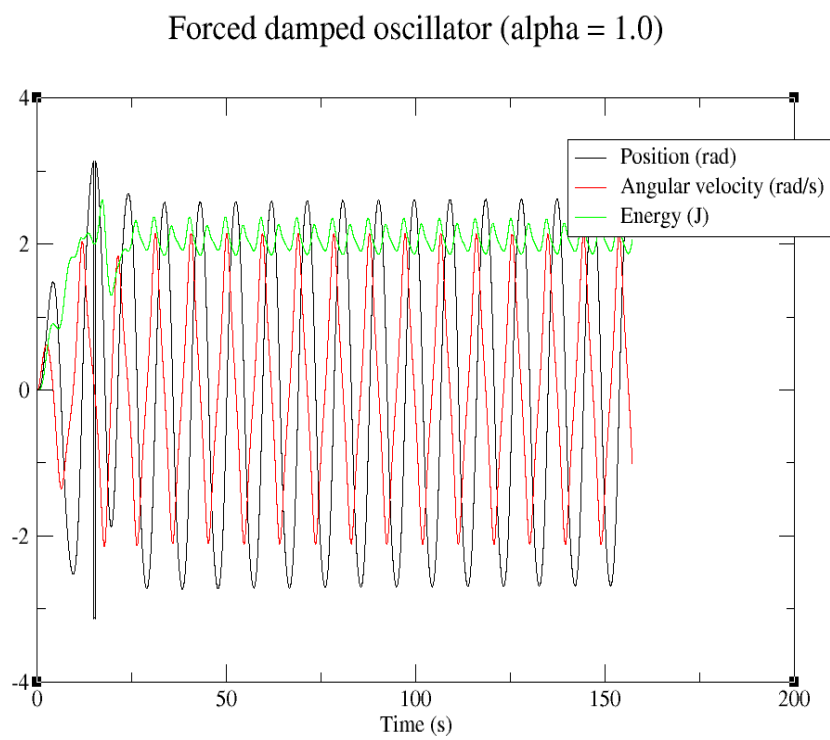


Figura 18: Posição, velocidade angular e energia de um pêndulo forçado amortecido com $\alpha = 1.0$. O sistema é periódico.

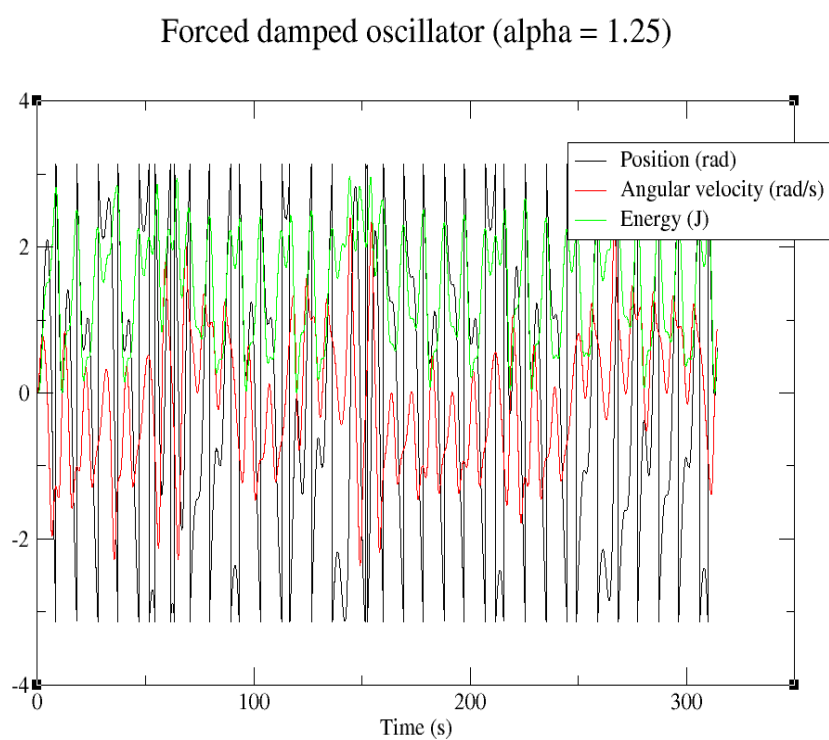


Figura 19: Posição, velocidade angular e energia de um pêndulo forçado amortecido com $\alpha = 1.25$. O sistema é caótico.

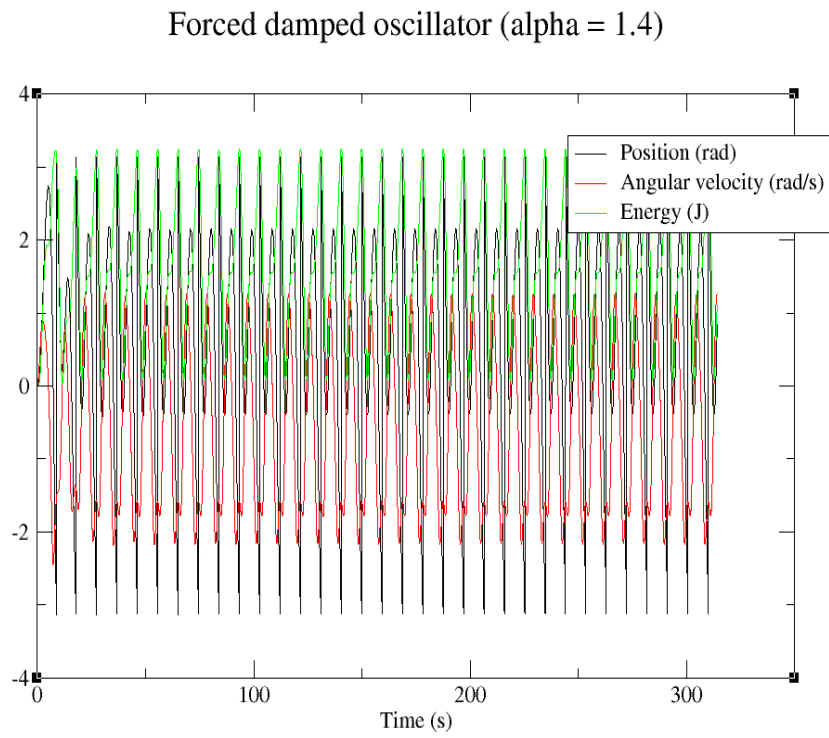


Figura 20: Posição, velocidade angular e energia de um pêndulo forçado amortecido com $\alpha = 1.4$. O sistema é caótico.

Forced damped oscillator ($\alpha = 1.5$)

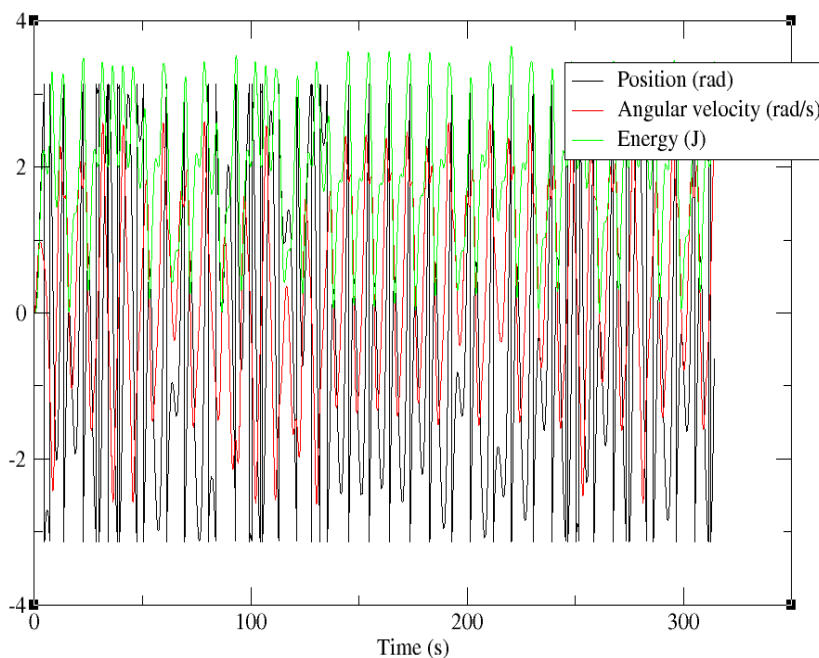


Figura 21: Posição, velocidade angular e energia de um pêndulo forçado amortecido com $\alpha = 1.5$. O sistema é caótico.

Em geral, os casos periódicos estão de acordo com o esperado, pois todos oscilam (após um transiente) com uma frequência igual a da força externa.

Vale a pena observar que na figura (20), há um caso interessante: o movimento parece periódico, mas na verdade é caótico. Isso está mostrada na figura (26), onde a curva que forma parece com uma elipse (caso periódico), mas não é.

4.2 Velocidade angular em função da posição.

Agora será plotado a velocidade como função da posição de cada um dos casos das figuras (16) a (21). Esses resultados estão nas figuras (23) a (28). É de se esperar que para os casos periódicos, elipses formarão (porque é como se o movimento final fosse regido inteiramente pela força externa, que é periódica, logo a velocidade e posição serão senos e cossenos multiplicados por fatores constantes. Isso resulta na equação de uma elipse). Na prática, isso ocorre (figuras 23 a 25), porém há traços no meio das elipses que representam os estados transientes antes de chegarem a ser periódicos.

O código fonte está na figura (22). É basicamente a primeira parte do código fonte da tarefa anterior. Foi apenas rodado a simulação até o tempo máximo e plotado os resultados.

```

implicit none
real*8, parameter :: dt = 1d-3
real*8, parameter :: gam = 5d-1
real*8, parameter :: v = 2d0/3d0
real*8, parameter :: alpha = 15d-1
real*8, parameter :: pi = dacos(-1d0)
real*8, parameter :: T_max = 100*pi
real*8 :: T, theta_i, omega_i, next_theta, next_omega
real*8 :: period, last_zero
integer :: oscillations, first
open(unit=1,file="tarefa-4a-saida-11.dat")
  T = 0
  theta_i = 0
  omega_i = 0
  do while(T .le. T_max) ! until T_max/2 for period motions
    if (theta_i .lt. -pi) then
      theta_i = theta_i + 2*pi
    else if (theta_i .gt. pi) then
      theta_i = theta_i - 2*pi
    end if
    write(1,*) T, theta_i, omega_i, E(theta_i,omega_i)
    next_omega = omega_i + (alpha*d sin(v*T) - gam*omega_i -
&d sin(theta_i))*dt
    next_theta = theta_i + next_omega*dt
    theta_i = next_theta
    omega_i = next_omega
    T = T + dt
  end do
close(1)

```

Figura 22: Código fonte da tarefa 4b (velocidade em função do tempo de um pêndulo forçado amortecido). (ignore comentários).

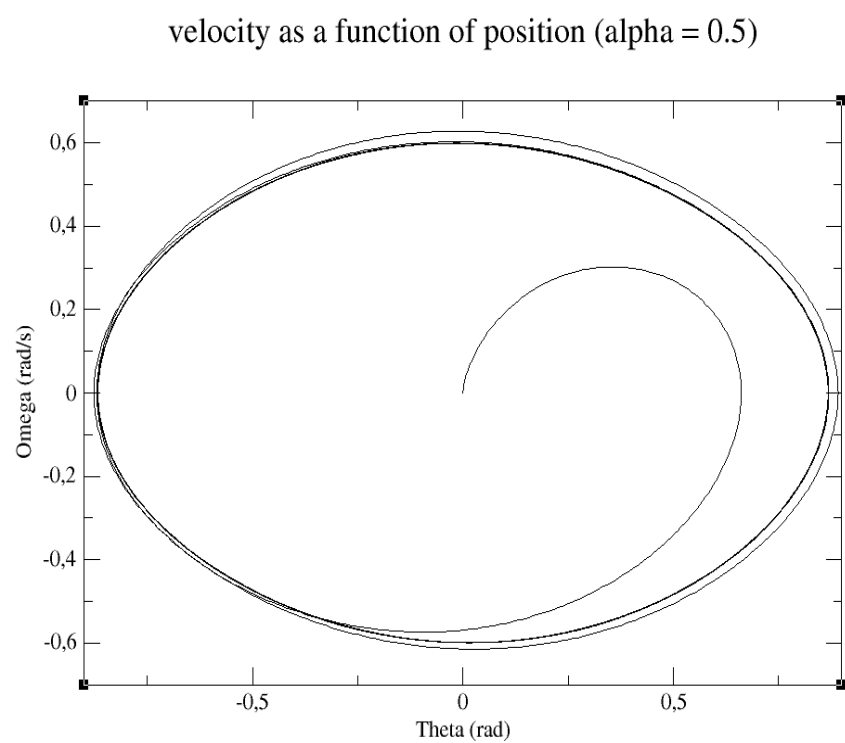


Figura 23: Velocidade em função do tempo de um pêndulo forçado amortecido com $\alpha = 0.5$. O sistema é periódico.

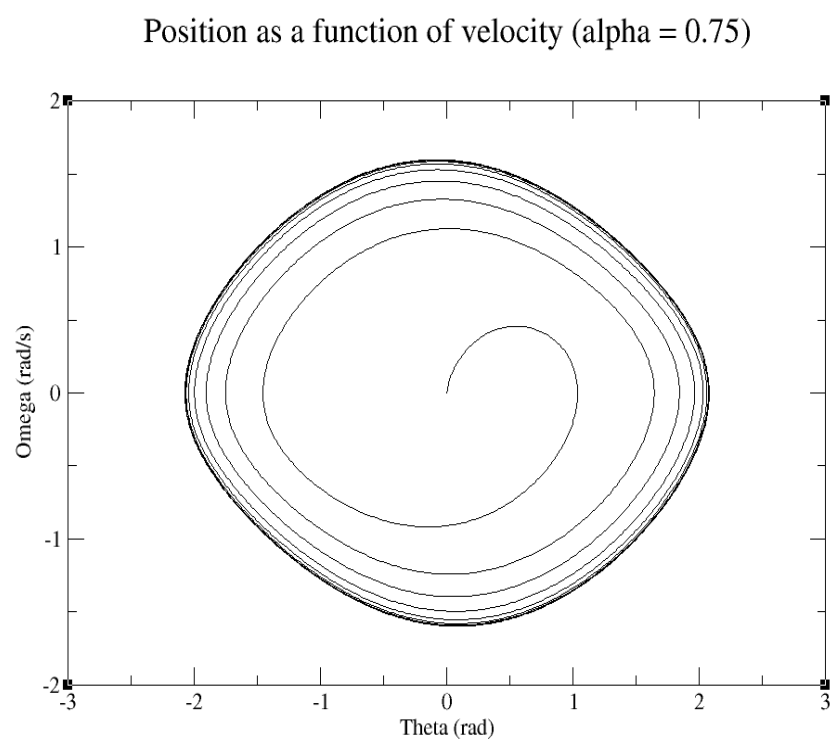


Figura 24: Velocidade em função do tempo de um pêndulo forçado amortecido com $\alpha = 0.75$. O sistema é periódico.

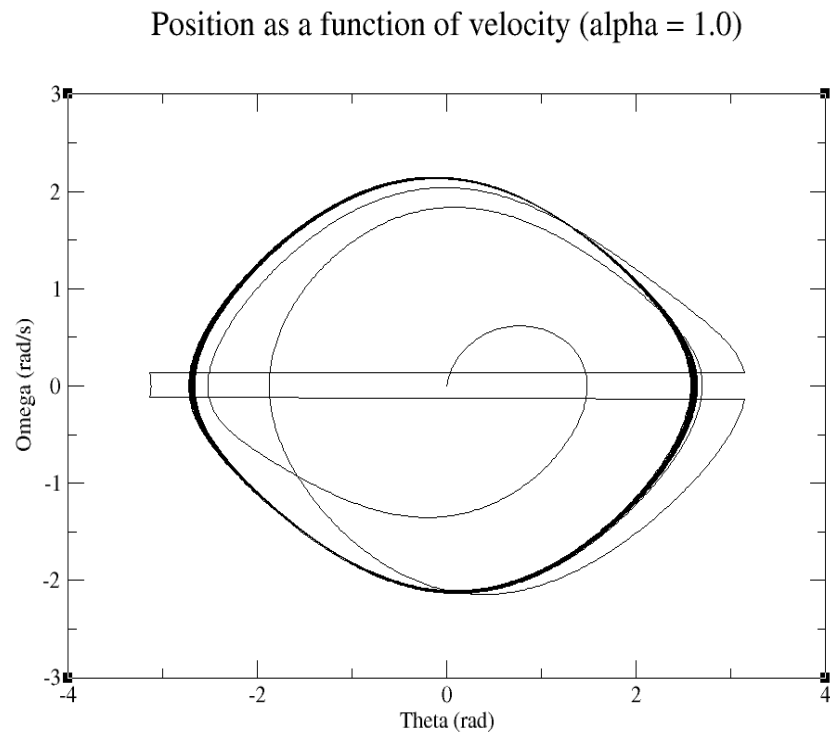


Figura 25: Velocidade em função do tempo de um pêndulo forçado amortecido com $\alpha = 1.0$. O sistema é periódico.

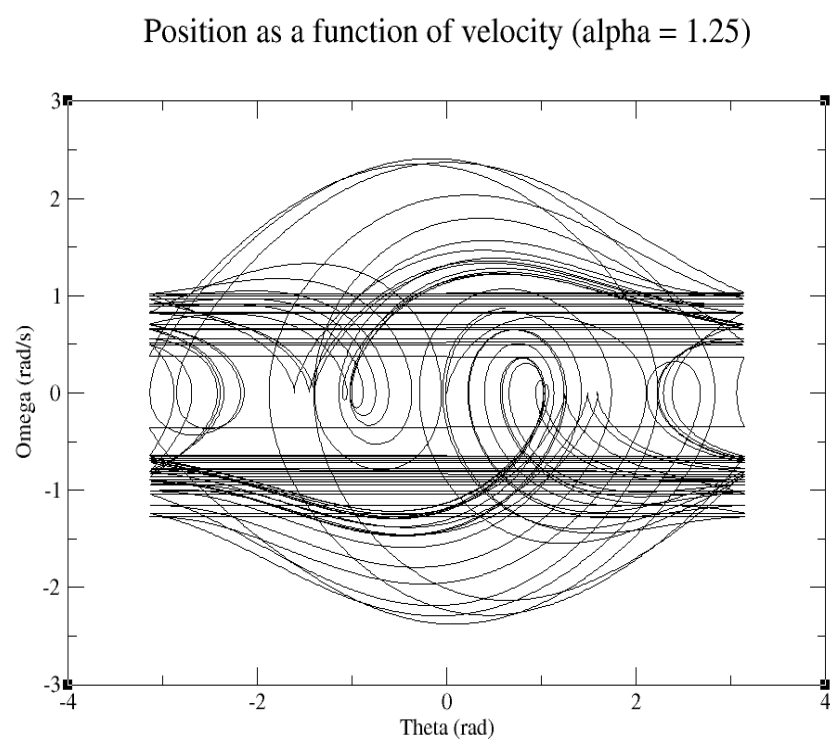


Figura 26: Velocidade em função do tempo de um pêndulo forçado amortecido com $\alpha = 1.25$. O sistema é caótico.

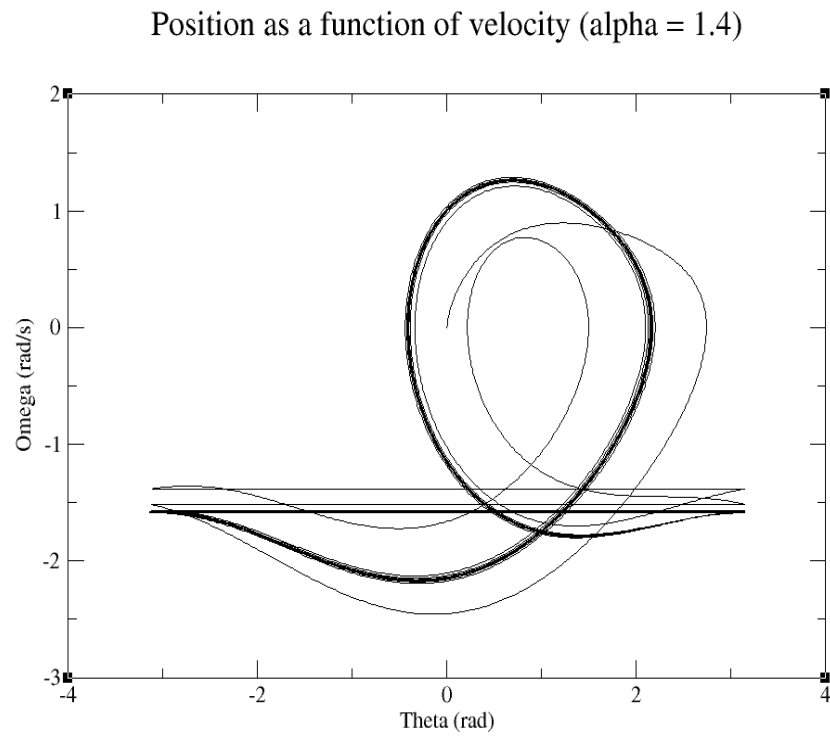


Figura 27: Velocidade em função do tempo de um pêndulo forçado amortecido com $\alpha = 1.4$. O sistema é caótico.

Position as a function of velocity (alpha = 1.5)

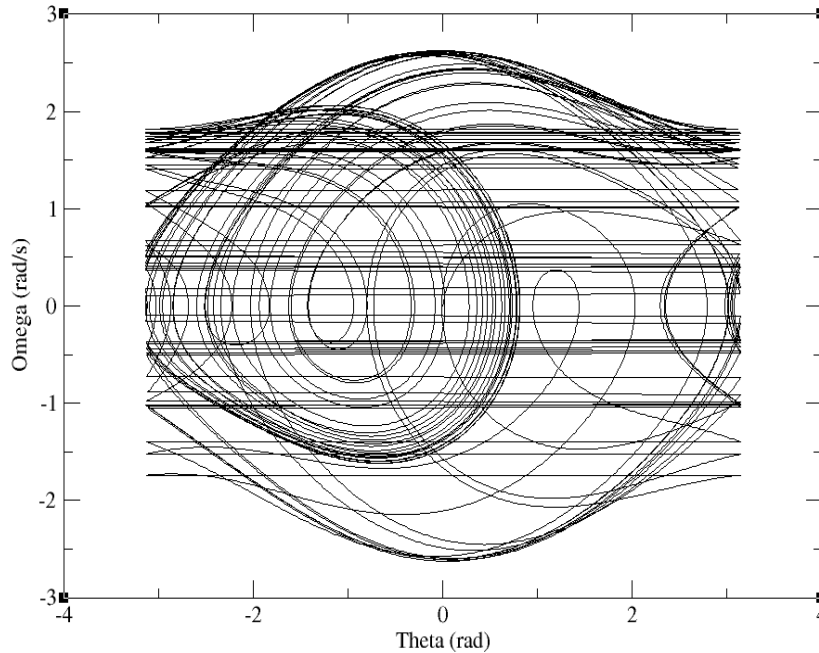


Figura 28: Velocidade em função do tempo de um pêndulo forçado amortecido com $\alpha = 1.5$. O sistema é caótico.

4.3 Seções de Poincaré.

Nessa parte da tarefa, foi plotado as seções de Poincaré do sistema em questão para os diferentes valores de alpha. O código fonte da tarefa está na figura (29). As seções de Poincaré foram calculados escrevendo em um arquivo a posição e velocidade do sistema para tempos múltiplos de $2\pi/\nu = 3\pi$. Isso foi feito através da função intrínseca do fortran 77 : "dmod", que calcula o resto da divisão do primeiro argumento (real) pelo segundo argumento (real também). O resultado disso tem que ser menor que o incremento de tempo dado (0.001 segundos), porque essa será a precisão empregada (se for maior, então com certeza não será múltiplo).

Para os casos periódicos, as seções de Poincaré são plotados assim que o estado transiente for ultrapassado. Isso se garante rodando a simulação até um terço do tempo máximo permitido. Pelas figuras (16) a (18), isso é tempo suficiente. Para os casos não periódicos, as seções de Poincaré são plotadas imediatamente. Isso é feito ignorando o primeiro laço na figura (29) (por exemplo, fazendo "do while (T .le. 0)").

```

implicit none
real*8, parameter :: dt = 1d-3
real*8, parameter :: gam = 5d-1
real*8, parameter :: v = 2d0/3d0
real*8, parameter :: alpha = 125d-2
real*8, parameter :: pi = dacos(-1d0)
real*8, parameter :: T_max = 100*pi
real*8 :: T, theta_i, omega_i, next_theta, next_omega

T = 0
theta_i = 0
omega_i = 0
do while(T .le. T_max/3) ! reach steady state for periodic
    if (theta_i .lt. -pi) then
        theta_i = theta_i + 2*pi
    else if (theta_i .gt. pi) then
        theta_i = theta_i - 2*pi
    end if
    next_omega = omega_i + (alpha*d sin(v*T) - gam*omega_i -
&d sin(theta_i))*dt
    next_theta = theta_i + next_omega*dt
    theta_i = next_theta
    omega_i = next_omega
    T = T + dt
end do

! steady-state is achieved.
open(unit=1,file="tarefa-4c-saida-7.dat")
do while (T .le. T_max)
    if (theta_i .lt. -pi) then
        theta_i = theta_i + 2*pi
    else if (theta_i .gt. pi) then
        theta_i = theta_i - 2*pi
    end if
    ! precision is dictated by size of dt chosen
    if (dmod(T,3*pi) .le. 1e-3) then
        write(1,*) theta_i, omega_i
    end if
    next_omega = omega_i + (alpha*d sin(v*T) - gam*omega_i -
&d sin(theta_i))*dt
    next_theta = theta_i + next_omega*dt
    theta_i = next_theta
    omega_i = next_omega
    T = T + dt
end do
close(1)
end

```

Figura 29: Código fonte da tarefa 4c (cálculo das seções de Poincaré para um pêndulo forçado amortecido).

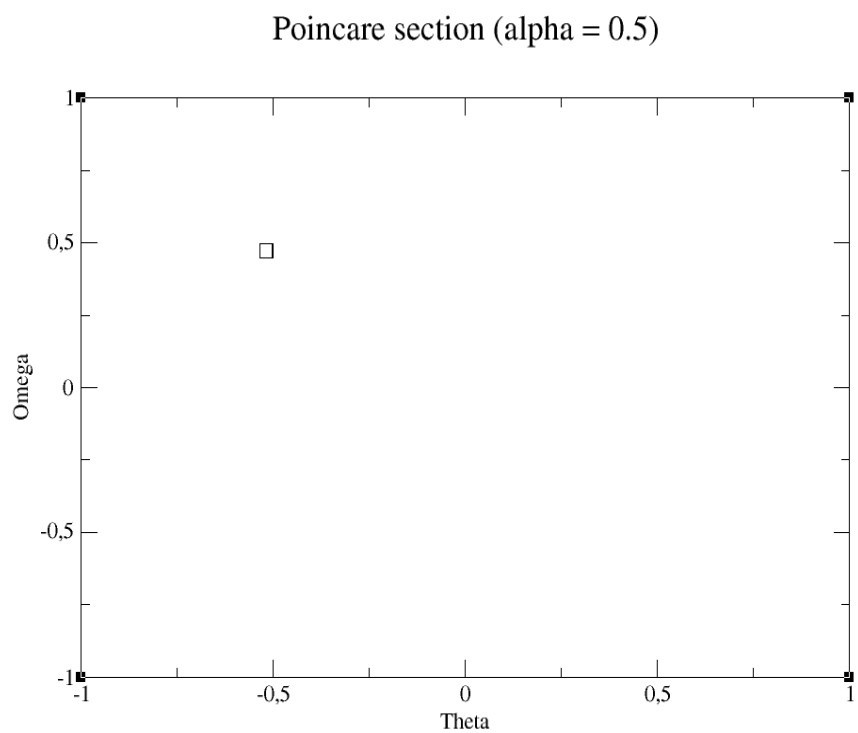


Figura 30: Seções de Poincaré para um pêndulo forçado amortecido com $\alpha = 0.5$. O movimento é periódico.

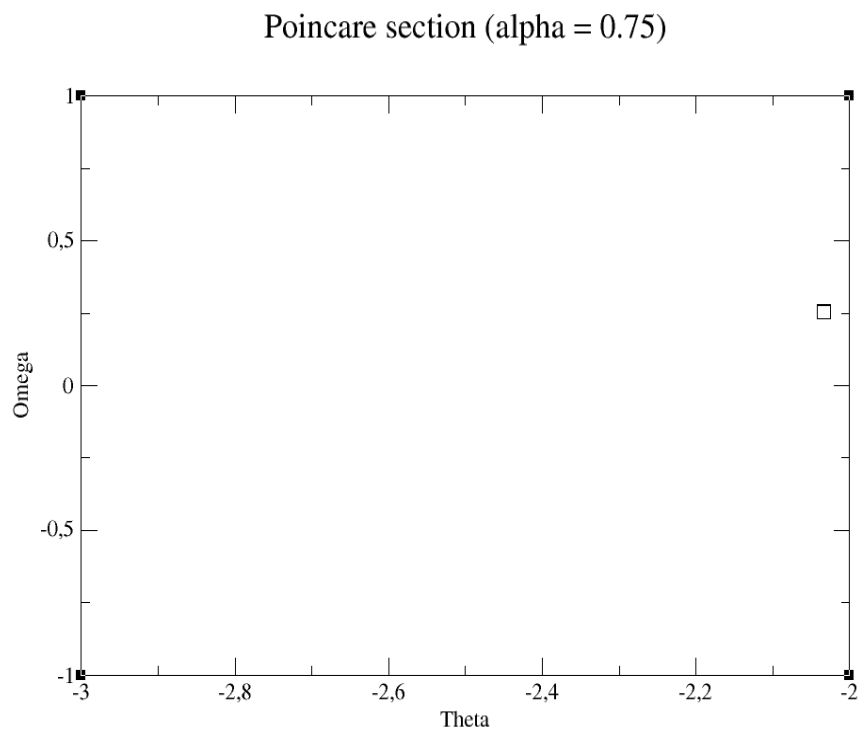


Figura 31: Seções de Poincaré para um pêndulo forçado amortecido com $\alpha = 0.75$. O movimento é periódico.

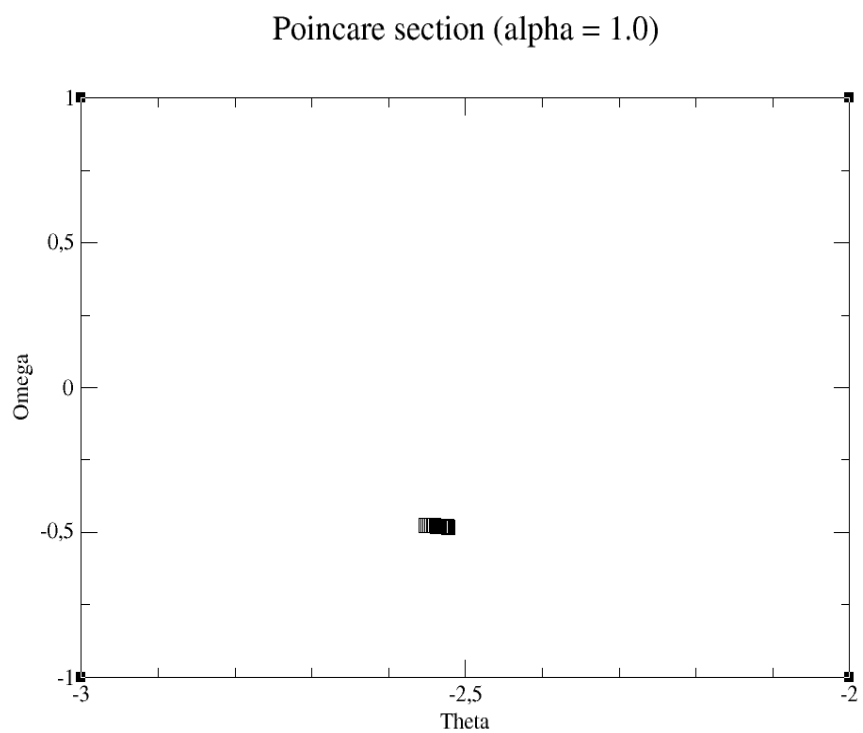


Figura 32: Seções de Poincaré para um pêndulo forçado amortecido com $\alpha = 1.0$. O movimento é periódico.

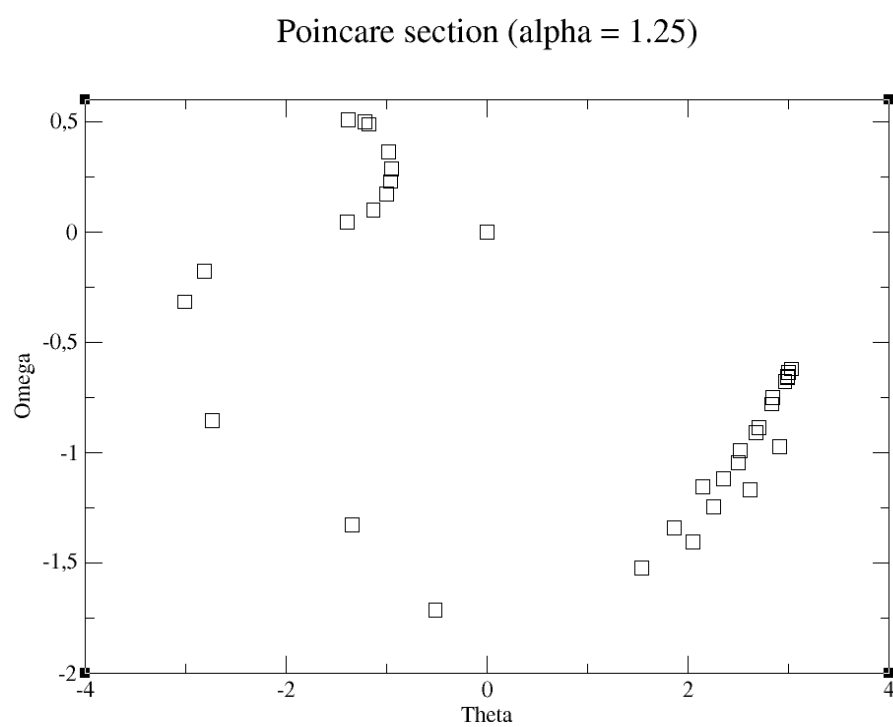


Figura 33: Seções de Poincaré para um pêndulo forçado amortecido com $\alpha = 1.25$. O movimento é caótico.

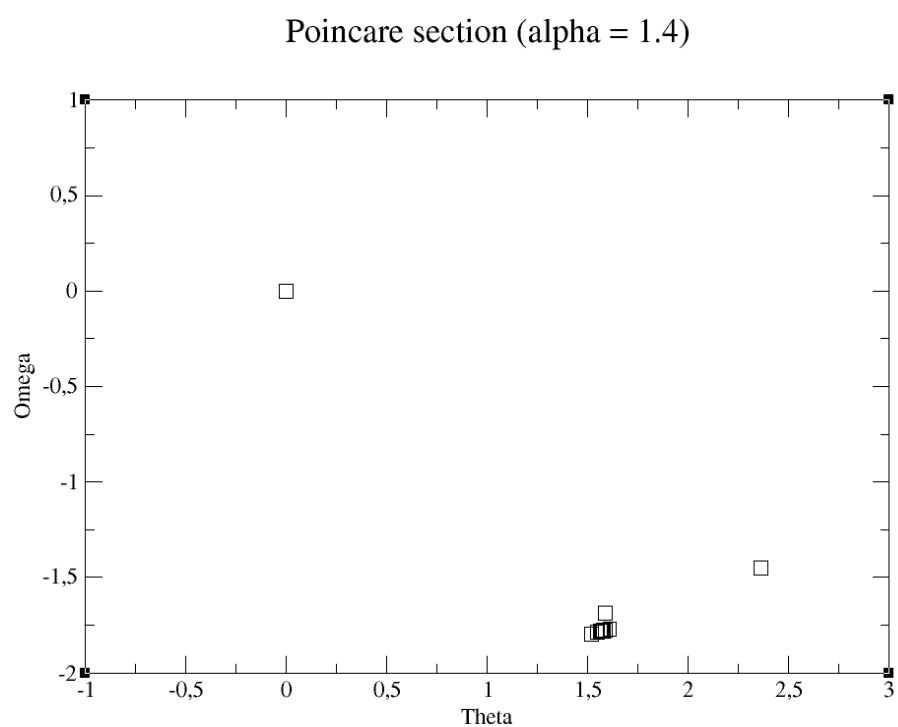


Figura 34: Seções de Poincaré para um pêndulo forçado amortecido com $\alpha = 1.4$. O movimento é caótico.

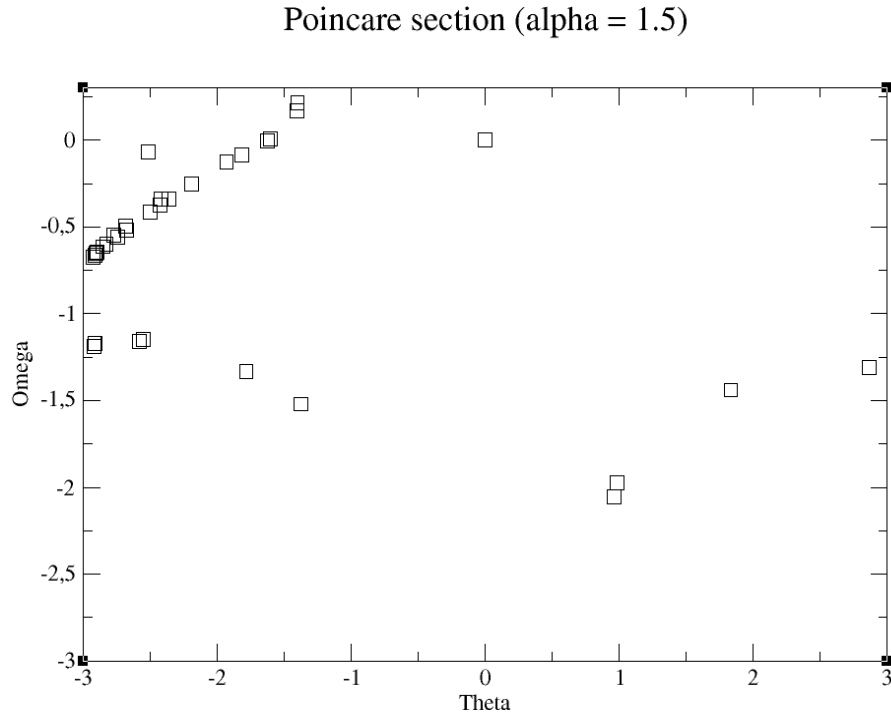


Figura 35: Seções de Poincaré para um pêndulo forçado amortecido com $\alpha = 1.5$. O movimento é caótico.

Convém observar que para os casos periódicos, as seções de Poincaré se reduzem a um ponto. Isso se explica pelo fato de que o movimento é regido essencialmente pela força externa, logo para tempos múltiplos do período dessa força externa, a posição do pêndulo será sempre o mesmo. Para os casos caóticos, as seções de Poincaré se reduzem a fractais.

4.4 Expoente de Lyapunov.

Por fim, será calculado o expoente de Lyapunov do sistema em questão para cada valor de α utilizado até agora. Esses valores estão plotados na tabela 1.

O código fonte está nas figuras (36) a (38). O problema foi dividido em três partes: primeiro foi rodado a simulação do pêndulo para um dado valor de α com velocidade inicial nula e velocidade inicial não nula (igual a 10^{-2} para o caso periódico ou 10^{-5} para o caso caótico). A posição e velocidade para o sistema com velocidade inicial não nula foi escrita em um arquivo de saída assim como a diferença entre a posição e velocidade entre as duas simulações. Não foi plotado mais nada nas figuras (39) a (44) para evitar poluir a imagem. A segunda parte consiste em contar o período do sistema com velocidade inicial não nula (somente se for periódico) com o método realizado anteriormente. Depois, foi armazenado o logaritmo natural da posição e velocidade em um vetor para ser utilizado depois. A terceira parte consiste em usar os dados dos vetores para

fazer uma regressão linear entre os dados e assim, obter o expoente de Lyapunov para cada caso.

A explicação para esse método é o seguinte: para ajustar uma exponencial aos dados da diferença de fase e velocidade, será feita uma regressão linear sobre um papel mono-log:

$$\delta\theta = ae^{\lambda_\theta t} \iff \ln(\delta\theta) = \ln(a) + \lambda_\theta t$$

O coeficiente angular é o expoente de Lyapunov. Então basta usar a fórmula (7) de regressão linear:

$$\lambda_\theta = \frac{\sum_{i=1}^n (t_i - \hat{t})(\delta\theta_i - \hat{\delta\theta})}{\sum_{i=1}^n (t_i - \hat{t})^2} \quad (7)$$

Onde $\hat{t}$ e $\hat{\delta\theta}$ são as médias dos dados. No código fonte, foi feita uma simplificação para agilizar o código: o denominador da equação (7) foi reduzido para :

$$2 \times 10^{-6} \times \frac{(N)(N+1)(2N+1)}{6}$$

Onde N é número do total de intervalos de tempos dividido por dois. Essa simplificação vem do fato de que os tempos são uma progressão aritmética, então um único intervalo de tempo é escrito como $t_i = t_0 + dt(i-1)$. Logo a média do intervalo total é:

$$\frac{\sum_{i=1}^n t_0 + dt(i-1)}{n} = t_0 + dt(n-1)/2$$

Nesse caso, esse valor será $\hat{t} = 0 + dt(\lfloor 100 \times \pi \rfloor / dt - 1)/2 = 157.079$. Além disso, o denominador da equação (7) pode ser escrito como:

$$\sum_{i=1}^n (t_i - \hat{t})^2 = 2(t_0 - \hat{t})^2 + 2(t_1 - \hat{t})^2 + \dots + 2(t_{\lfloor n/2 \rfloor} - \hat{t})^2$$

Então, de trás para frente,

$$2(t_{\lfloor n/2 \rfloor} - \hat{t})^2 + \dots + 2(t_1 - \hat{t})^2 + 2(t_0 - \hat{t})^2 = 0 + 2(1)10^{-6} + 2(4)10^{-6} + 2(9)10^{-6} + \dots + 2(N^2)10^{-6}$$

Onde N é número do total de intervalos de tempos dividido por dois. Como a soma dos quadrados de números naturais é dado por: $\sum_{i=1}^n n^2 = \frac{(n)(n+1)(2n+1)}{6}$, então obtém-se a simplificação dada anteriormente. Isso está na figura (36) como uma expressão parâmetro.

Além disso, na figura (38), nos casos periódicos de movimento, essa fórmula simplificada foi dividido por 2 porque foi tomado apenas a segunda metade dos valores dos intervalos de tempo.

Os valores de expoentes de Lyapunov estão plotados na tabela 1. Esses valores fazem sentido porque $\lambda_\theta \approx \lambda_\omega$. Além disso, para os casos periódicos (primeiros três valores de α), o expoente está muito próximo do zero e negativo, o que significa que a exponencial que melhor se ajusta aos dados é praticamente constante ao longo do eixo x e tende a diminuir (o que faz sentido pois a força externa acaba sendo o agente principal pelo movimento após um certo intervalo de tempo). Para os casos caóticos, o expoente de Lyapunov é relativamente pequeno, porém positivo, o que significa que há pouca diferença entre os dois

sistemas, exceto quando o tempo fica grande . A única exceção é $\alpha = 1.4$, que embora seja um movimento caótico, o meu programa não retornou um expoente de Lyapunov coerente. Talvez isso seja porque esse valor em específico se assemelha com um movimento periódico, conforme foi apontado anteriormente.

α	λ_θ	λ_ω
0.5	-2.10×10^{-6}	-2.10×10^{-6}
0.75	-2.34×10^{-2}	-2.35×10^{-2}
1.0	-1.84×10^{-3}	-1.86×10^{-3}
1.25	3.72×10^{-2}	3.68×10^{-2}
1.4	-4.43×10^{-2}	-4.48×10^{-2}
1.5	5.13×10^{-2}	5.06×10^{-2}

Tabela 1: Expoente de Lyapunov para diferentes valores de alpha usados até agora. λ_θ é o expoente de Lyapunov para a diferença de fase entre o sistema com velocidade inicial nula e velocidade inicial igual a 10^{-2} ou 10^{-5} . λ_ω é o expoente de Lyapunov para a diferença de velocidade entre o sistema com velocidade inicial nula e velocidade inicial igual a 10^{-2} ou 10^{-5} .

```

implicit none
real*8, parameter :: dt = 1d-3
real*8, parameter :: gam = 5d-1
real*8, parameter :: v = 2d0/3d0
real*8, parameter :: alpha = 5d-1
real*8, parameter :: pi = dacos(-1d0)
real*8, parameter :: T_max = 100*pi
integer, parameter :: T_total = int(T_max/dt)
real*8, parameter :: T_average = 1d-3*(T_total - 1)/2
real*8, parameter :: CONST = 2*1d-6*(int(T_total/2))*
&(int(T_total/2) + 1)*(2*int(T_total/2)+1)/6
real*8 :: T, theta_i, omega_i, next_theta, next_omega
real*8 :: ti, oi, nti, noi
real*8 :: period, last_zero, y1_avrg, y2_avrg, Lyapunov2
real*8 :: y1(T_total), y2(T_total), Lyapunov1, initial_t
integer :: oscillations, first, i, j

open(unit=1,file="tarefa-4d-saida-1.dat")
T = 0
theta_i = 0
omega_i = 1d-2 ! 1d-5 for chaotic motions
ti = 0
oi = 0
do while(T .le. T_max)
  if (theta_i .lt. -pi) then
    theta_i = theta_i + 2*pi
  else if (theta_i .gt. pi) then
    theta_i = theta_i - 2*pi
  end if
  if (ti .lt. -pi) then
    ti = ti + 2*pi
  else if (ti .gt. pi) then
    ti = ti - 2*pi
  end if
  write(1,*) T, theta_i, omega_i, dabs(ti-theta_i),
&dabs(oi-omega_i)
  ! NEW trajectory
  next_omega = omega_i + (alpha*d sin(v*T) - gam*omega_i -
&d sin(theta_i))*dt
  next_theta = theta_i + next_omega*dt
  theta_i = next_theta
  omega_i = next_omega
  ! OLD trajectory
  noi = oi + (alpha*d sin(v*T) - gam*oi - d sin(ti))*dt
  nti = ti + noi*dt
  ti = nti
  oi = noi
  T = T + dt
end do
close(1)

```

Figura 36: Código fonte da tarefa 4d (cálculo do expoente de Lyapunov para o sistema pêndulo forçado amortecido). Parte 1: Simulação para plotar em gráfico.

```

! steady-state is achieved. Run simulations, do linear regression.
oscillations = 0
first = 1
T = 0 ! ACTIVATED ONLY FOR CHAOTIC MOTIONS
theta_i = 0 !...
omega_i = 1d-5 !...
ti = 0 !...
oi = 0 !...
initial_t = T
j = 1 ! indexing for y()
do while (T .le. T_max)
  if (theta_i .lt. -pi) then
    theta_i = theta_i + 2*pi
  else if (theta_i .gt. pi) then
    theta_i = theta_i - 2*pi
  end if
  if (ti .lt. -pi) then
    ti = ti + 2*pi
  else if (ti .gt. pi) then
    ti = ti - 2*pi
  end if

  next_omega = omega_i + (alpha*d*sin(v*T) - gam*omega_i -
&d*sin(theta_i))*dt
  next_theta = theta_i + next_omega*dt
  if (theta_i * next_theta .le. 0) then ! theta is zero
    if (first .eq. 1) then ! start counting
      first = 0
      period = T
    else
      oscillations = oscillations + 1
      last_zero = T
    end if
  end if
  theta_i = next_theta
  omega_i = next_omega
  noi = oi + (alpha*d*sin(v*T) - gam*oi -
&d*sin(ti))*dt
  nti = ti + noi*dt
  ti = nti
  oi = noi
  y1(j) = dlog(dabs(theta_i - ti) + 1d-15) ! to not have dlog(0)
  y2(j) = dlog(dabs(omega_i - oi) + 1d-15)
  j = j + 1
  T = T + dt
end do
! time from 2 zeros is half period
period = 2*(last_zero - period) / oscillations
print*, "Period: ", period

```

Figura 37: Código fonte da tarefa 4d (cálculo do expoente de Lyapunov para o sistema pêndulo forçado amortecido). Parte 2: Cálculo do período de movimentos periódicos e criação do vetor de dados para regressão linear.

```

! linear regression to find Lyapunov exponent
! average of differences
y1_avrg = 0
y2_avrg = 0
do i = 1,j
    y1_avrg = y1_avrg + y1(i)
    y2_avrg = y2_avrg + y2(i)
end do
y1_avrg = y1_avrg/j
y2_avrg = y2_avrg/j

! linear regression
Lyapunov1 = 0
Lyapunov2 = 0
if (oscillations .eq. 0) then ! chaotic
    do i = 1,j
        Lyapunov1 = Lyapunov1 + (initial_t - T_average)
        &*(y1(i) - y1_avrg)
        Lyapunov2 = Lyapunov2 + (initial_t - T_average)
        &*(y2(i) - y2_avrg)
        initial_t = initial_t + dt
    end do
    print*, "lyapunov of theta: ", Lyapunov1/CONST
    print*, "Lyapunov of omega: ", Lyapunov2/CONST
else ! periodic
    do i = 1,j
        Lyapunov1 = Lyapunov1 + (initial_t - T_average)
        &*(y1(i) - y1_avrg)
        Lyapunov2 = Lyapunov2 + (initial_t - T_average)
        &*(y2(i) - y2_avrg)
        initial_t = initial_t + dt
    end do
    print*, "lyapunov of theta: ", Lyapunov1/(CONST/2)
    print*, "Lyapunov of omega: ", Lyapunov2/(CONST/2)
end if
end

```

Figura 38: Código fonte da tarefa 4d (cálculo do expoente de Lyapunov para o sistema pêndulo forçado amortecido). Parte 3: Regressão linear.

Initial velocity = $1e-2$ / $\alpha = 0.5$

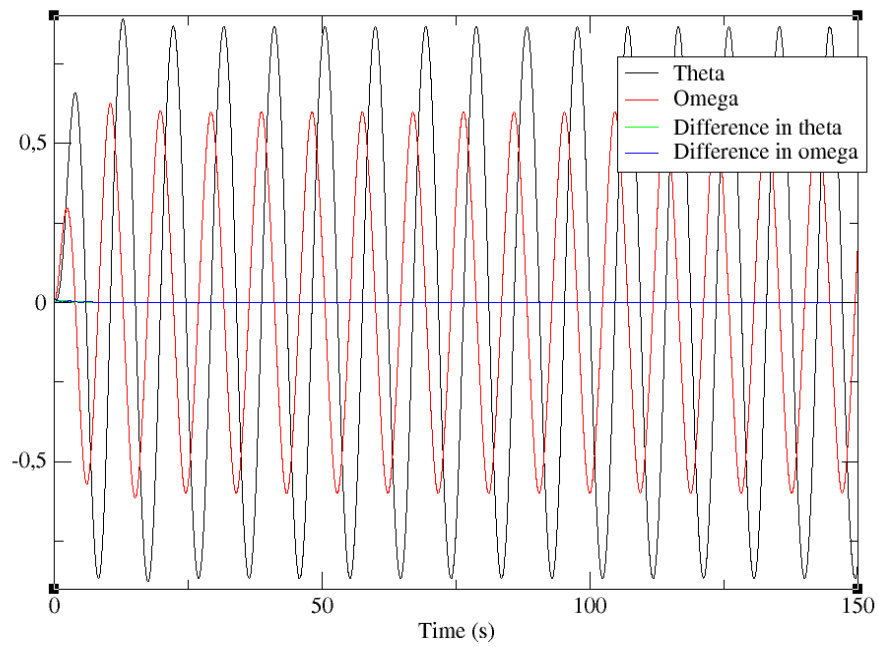


Figura 39: Pêndulo forçado amortecido sujeito a duas simulações ao mesmo tempo: uma com velocidade inicial nula e outra com velocidade inicial de 10^{-2} . Estão representados a posição e velocidade do sistema para a velocidade inicial não nula e a diferença de fase entre os dois sistemas.

Initial velocity = $1e-2 / \alpha = 0.75$

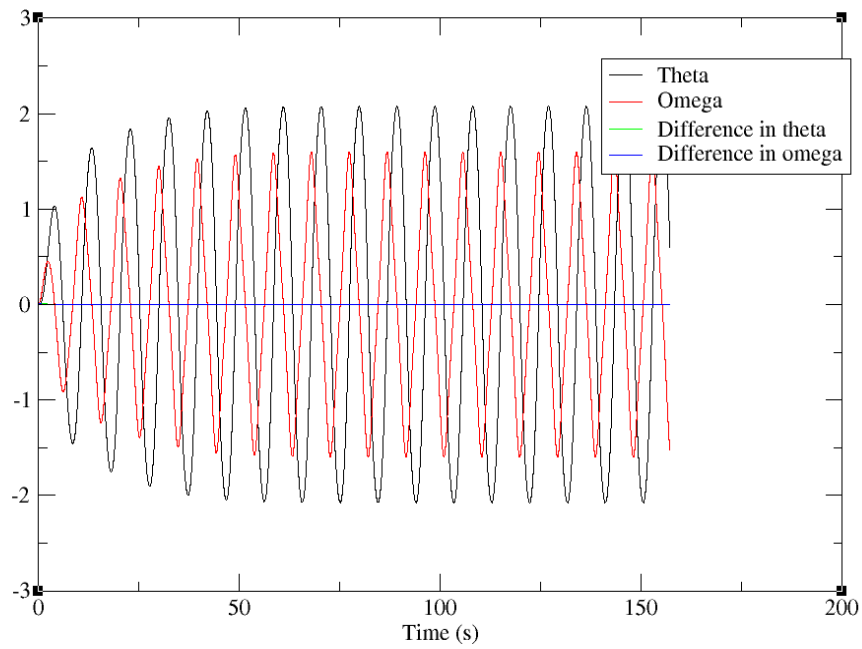


Figura 40: Pêndulo forçado amortecido sujeito a duas simulações ao mesmo tempo: uma com velocidade inicial nula e outra com velocidade inicial de 10^{-2} . Estão representados a posição e velocidade do sistema para a velocidade inicial não nula e a diferença de fase entre os dois sistemas.

Initial velocity = $1e-2$ / $\alpha = 1.0$

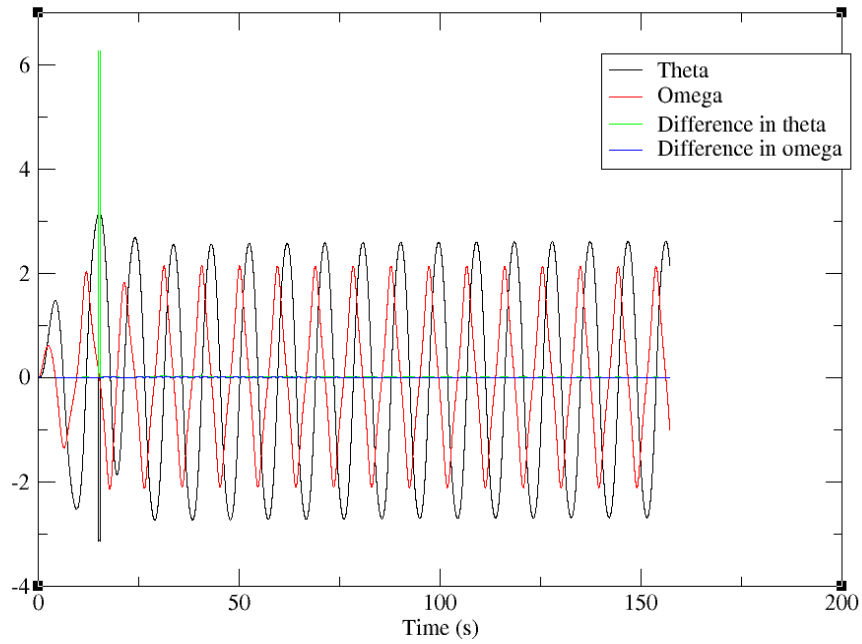


Figura 41: Pêndulo forçado amortecido sujeito a duas simulações ao mesmo tempo: uma com velocidade inicial nula e outra com velocidade inicial de 10^{-2} . Estão representados a posição e velocidade do sistema para a velocidade inicial não nula e a diferença de fase entre os dois sistemas.

Initial velocity = $1e-5$ / $\alpha = 1.25$

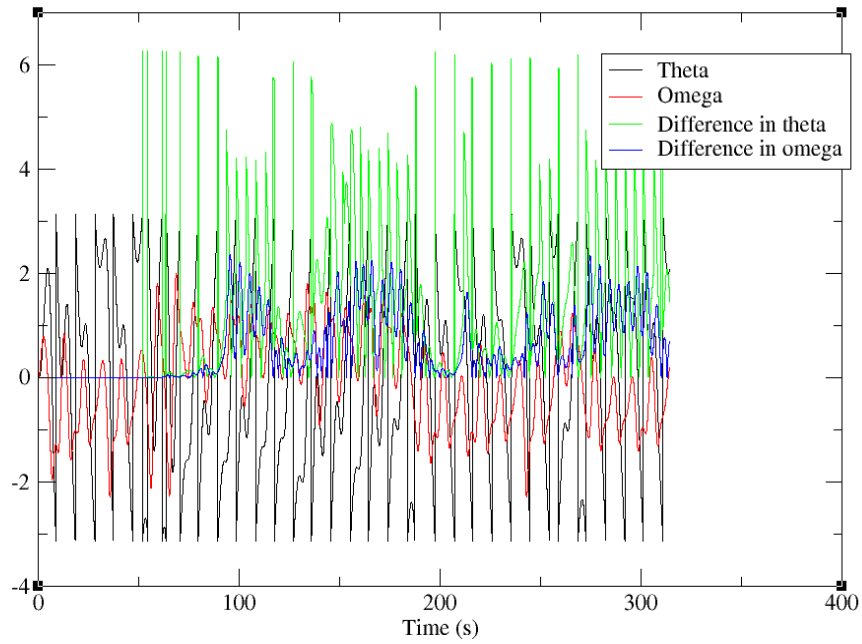


Figura 42: Pêndulo forçado amortecido sujeito a duas simulações ao mesmo tempo: uma com velocidade inicial nula e outra com velocidade inicial de 10^{-5} . Estão representados a posição e velocidade do sistema para a velocidade inicial não nula e a diferença de fase entre os dois sistemas.

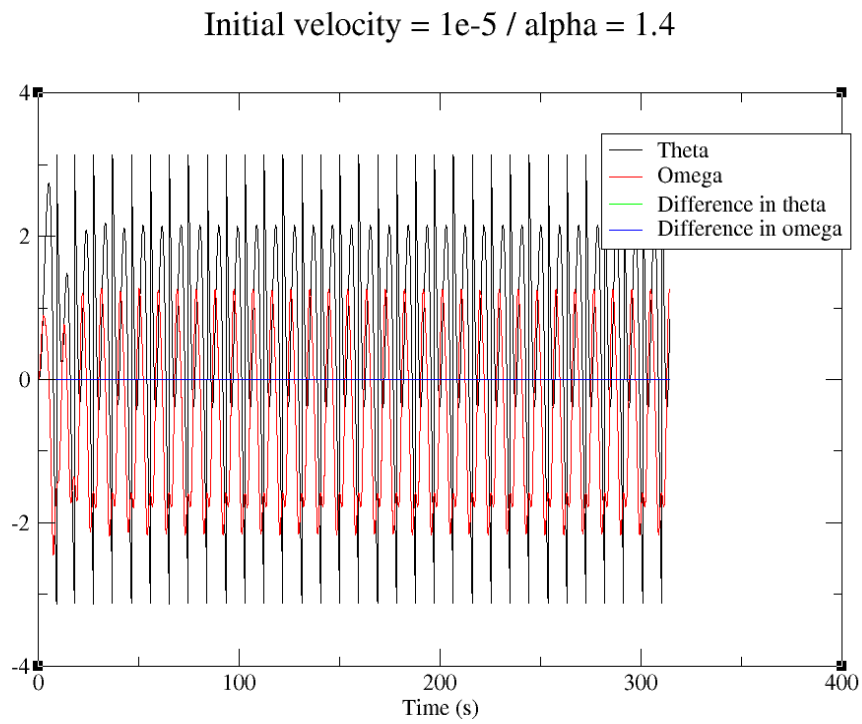


Figura 43: Pêndulo forçado amortecido sujeito a duas simulações ao mesmo tempo: uma com velocidade inicial nula e outra com velocidade inicial de 10^{-5} . Estão representados a posição e velocidade do sistema para a velocidade inicial não nula e a diferença de fase entre os dois sistemas.

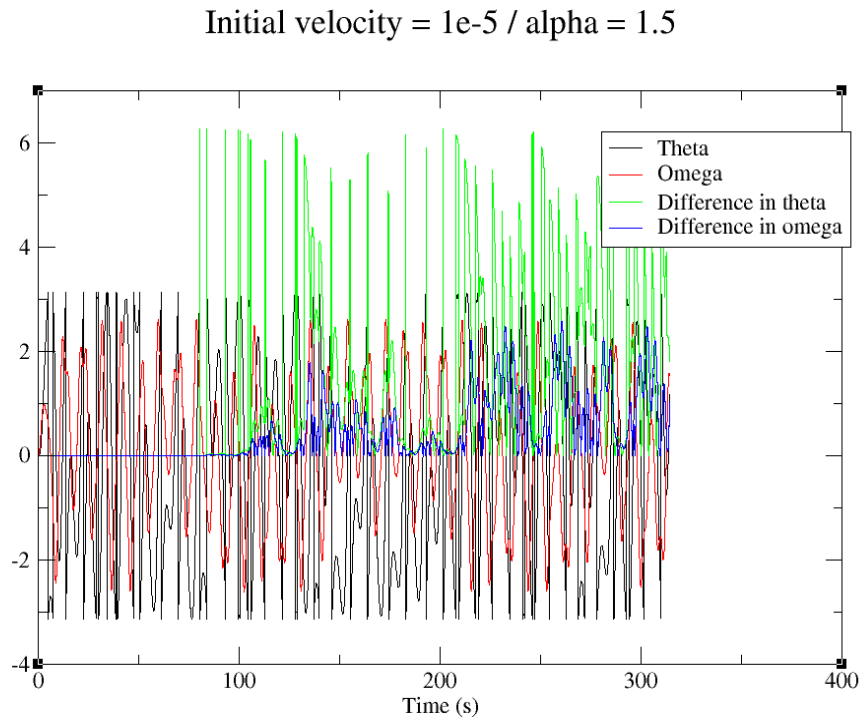


Figura 44: Pêndulo forçado amortecido sujeito a duas simulações ao mesmo tempo: uma com velocidade inicial nula e outra com velocidade inicial de 10^{-5} . Estão representados a posição e velocidade do sistema para a velocidade inicial não nula e a diferença de fase entre os dois sistemas.

Em geral, os resultados estão de acordo com o esperado. Para os casos periódicos, o período calculado foi de aproximadamente 9.42478 segundos em cada caso.